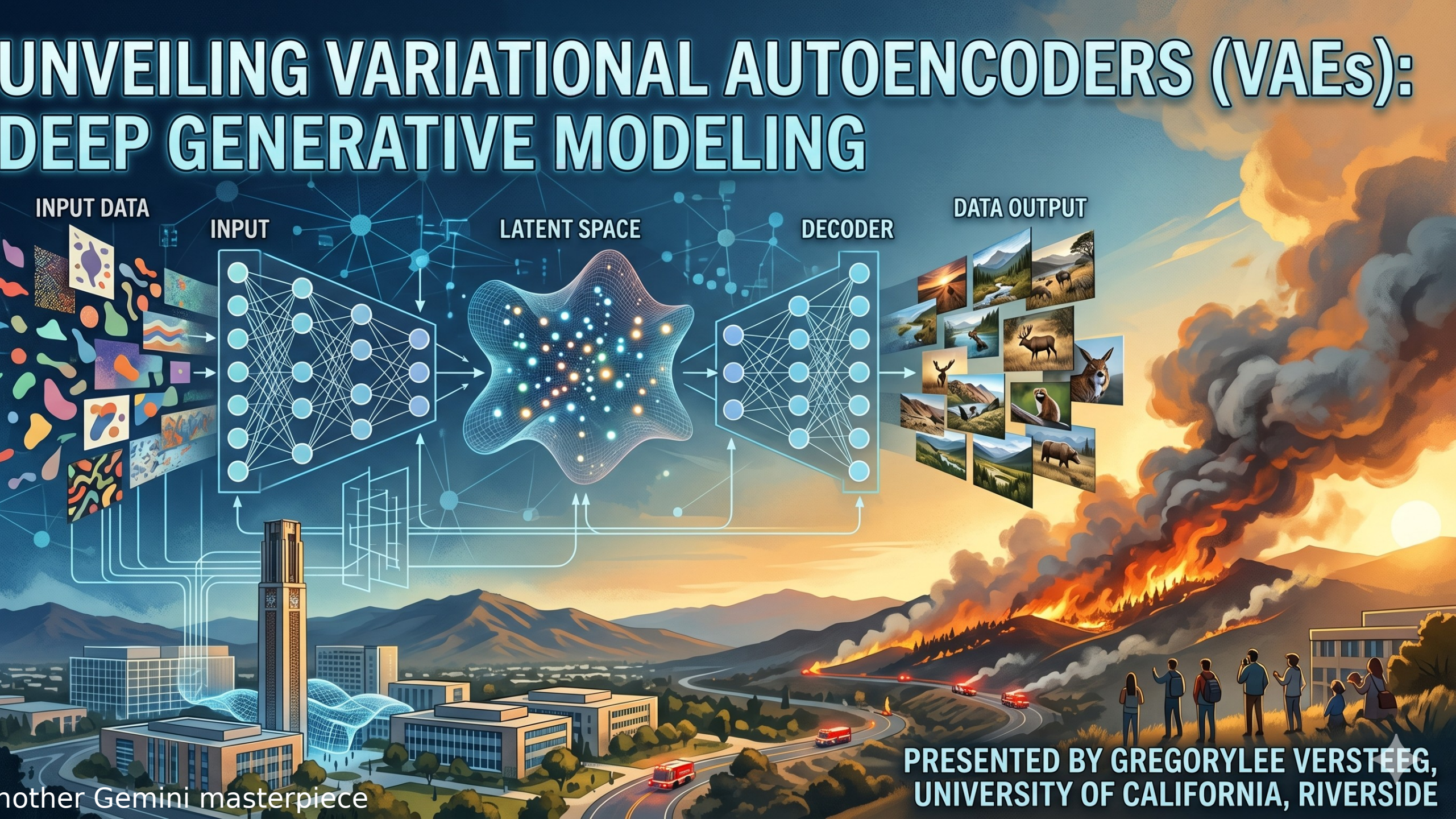
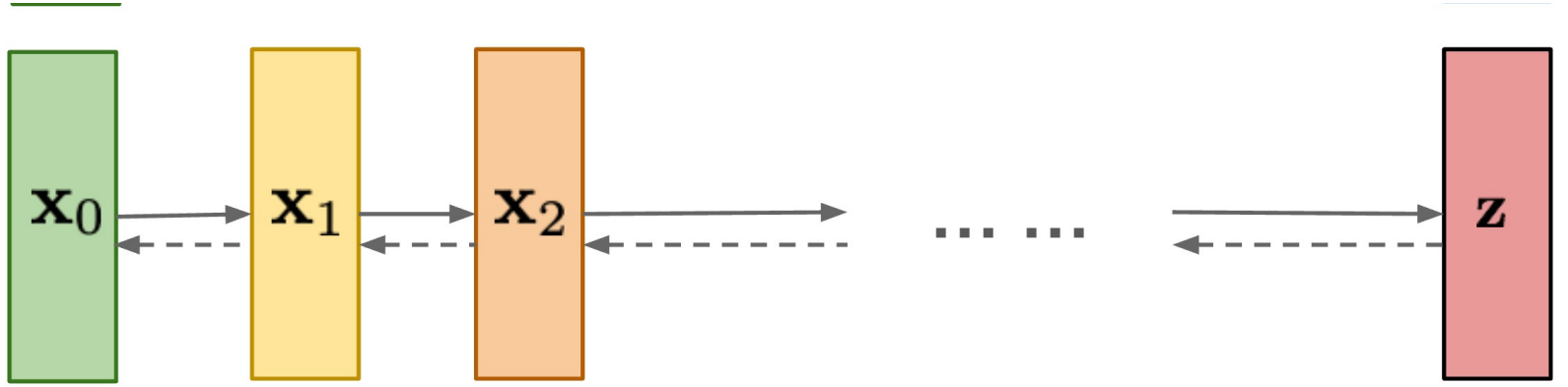
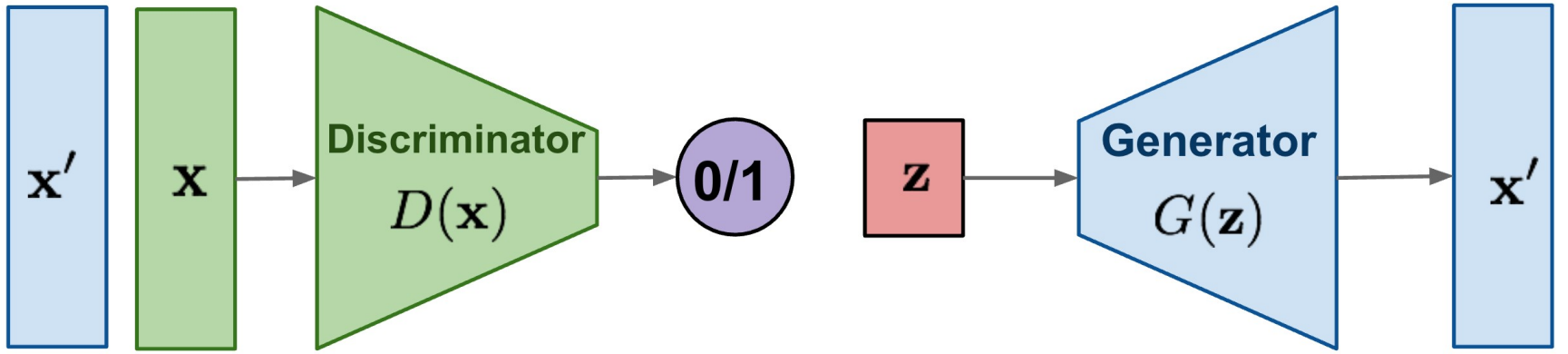

UNVEILING VARIATIONAL AUTOENCODERS (VAEs): DEEP GENERATIVE MODELING
INPUT DATA
INPUT
LATENT SPACE
DECODER
DATA OUTPUT
PRESENTED BY GREGORYLEE VERSTEEG, UNIVERSITY OF CALIFORNIA, RIVERSIDE
nother Gemini masterpiece

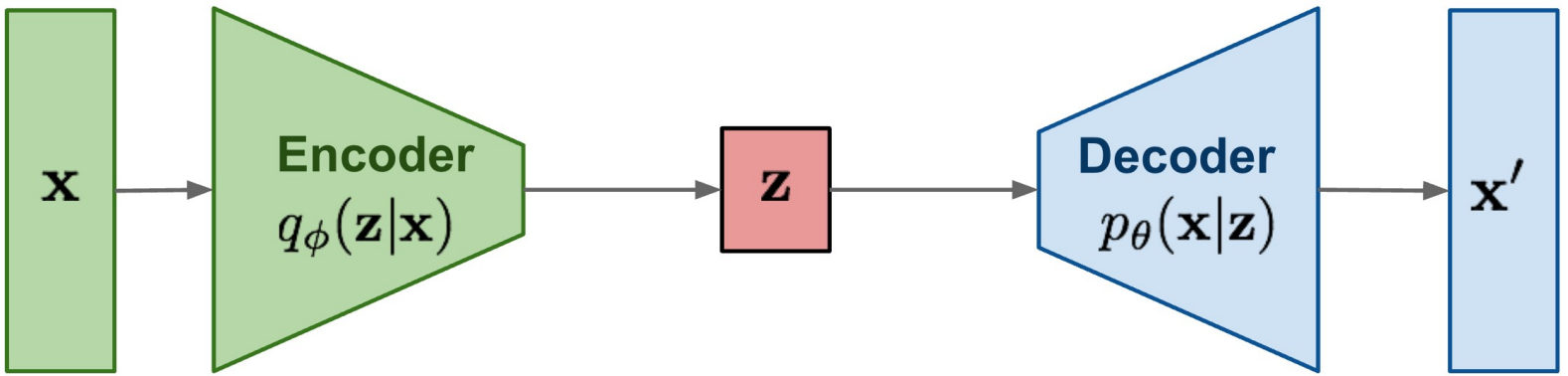
**Diffusion models:** Gradually add Gaussian noise and then reverse

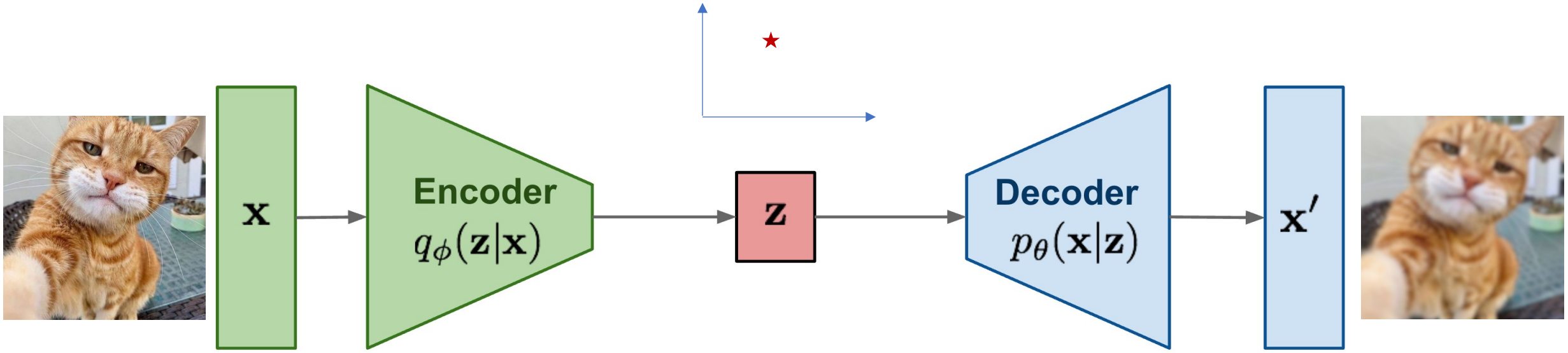



**GAN:** Adversarial training




**VAE:** maximize variational lower bound






- General autoencoder question: What should the loss function be?
- We're using probability distributions for the encoder-decoder…
  - What does it mean?
  - How to represent them?
  - How to *optimize* them?

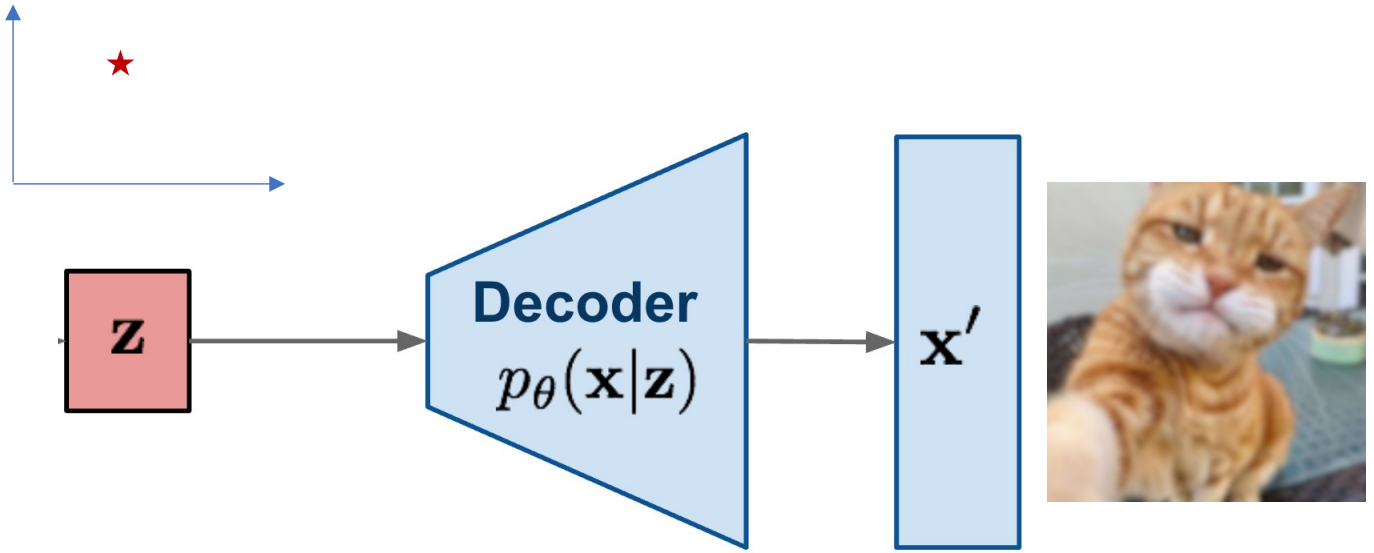
# Why Variational Auto-Encoders (VAE)?

Many useful ideas:

- “Variational distribution”
- Evidence Lower BOund (**ELBO)**
- “Reparametrization trick”

VAE and VQ-VAE still used for dimensionality reduction in latent diffusion (and probably for many other things, like audio)

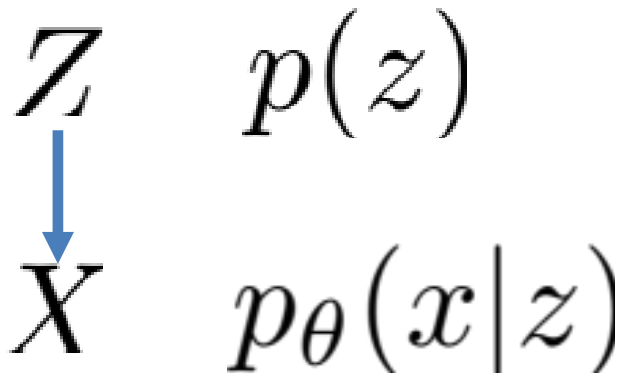
# A generative latent factor model (vs GAN)




(Draw on board)

# A ”generative” latent factor model

$$p_{(\theta)}(x, z) = p_\theta(x|z)p(z)$$




Example choices:

$$p(z) = \mathcal{N}(z; 0, \mathbf{I})$$

- $p(x|z) = N(x; \mu(z), I)$

# Variational optimization: make the data look likely under the generative model

- Take $x \sim q(x)$ (the data distribution)

$$\max_\theta E_{q(x)} \log p_\theta(x)$$

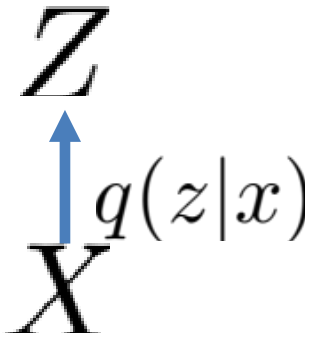
$$\max_\theta 1/N \sum_{\ell=1}^{N} \log p_\theta(x^{(\ell)})$$

- What is p(x)?

It's a hard integral…
Need many "z" values to estimate,
Each requires a neural net call

$$p(x) = \int dz \; p(x|z)p(z)$$

# Bayes rule hint for how to get p(x)?




- What if we use a *variational* approximation, q(z|x)?

$$D[q(z|x)||p(z|x)] = \mathbb{E}_{q(z|x)} \log \frac{q(z|x)}{p(z|x)} \geq 0$$

# One line ELBO derivation

The "gap" in the ELBO

True posterior

$$\log p(x) \geq \log p(x) - D_{KL}[q(z|x)||p(z|x)]$$

Variational approximation

*Show actual steps on boa*

$$= \mathbb{E}_{q(z|x)}[\log p(x|z)] - D_{KL}[q(z|x)||p(z)]$$

# Evidence Lower Bound (ELBO)

$$\log p(x) \geq \mathbb{E}_{q(z|x)}\left[\log p(x|z)\right] - D[q(z|x)||p(z)]$$

Our favorite!

Looks like a probabilistic auto-encoder reconstruction!

WTF!

For more about the ELBO: ”Variational Inference: A Review for Statisticians”: arXiv:1601.00670

# More complex bounds with extended state space dynamics

$$\log p(x) \geq \log p(x) - D_{KL}\left[q^{\mathrm{PROP}}(z_{\mathrm{ext}}|x)||p^{\mathrm{TGT}}(z_{\mathrm{ext}}|x)\right]$$

$$\mathbb{E}_{q^{\mathrm{PROP}}(z_{\mathrm{ext}}|x)}\left[\log \frac{p^{\mathrm{TGT}}(x, z_{\mathrm{ext}})}{q^{\mathrm{PROP}}(z_{\mathrm{ext}}|x)}\right]$$

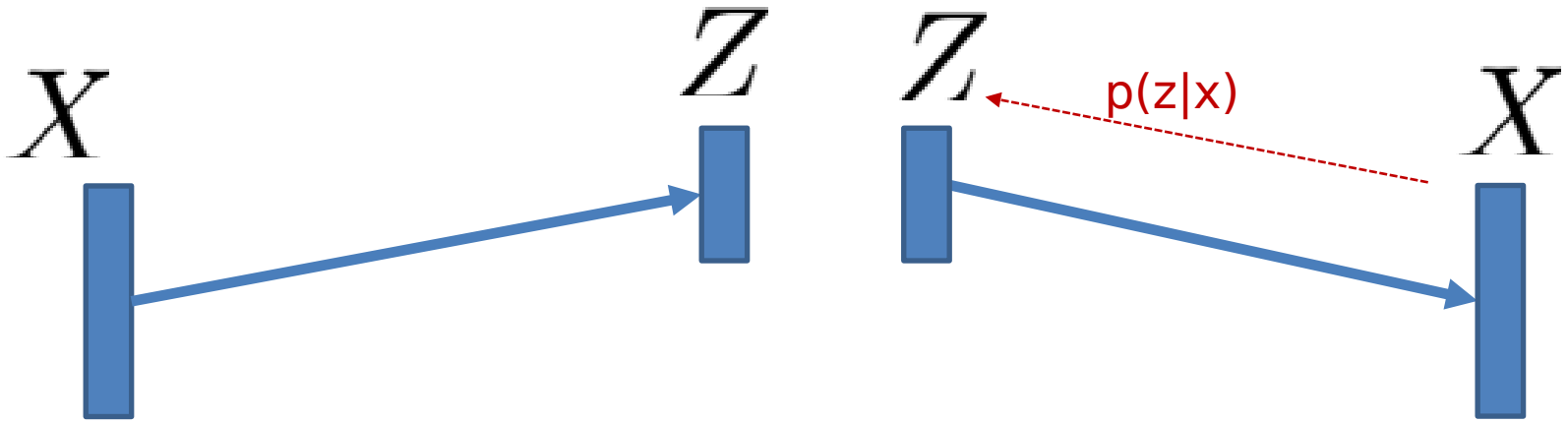
Sample crazy dynamics, multiple samples per point

Use tricks in here, to get importance weighted autoencoder bounds, AIS bounds, and generalizations

(Imans, Huang* et. al. “Improving MI Estimation using Annealed and Energy Based Bounds”, ICLR 2022.)

# Back to regular VAE

$$\log p(x) \geq \mathbb{E}_{q(z|x)}\left[\log p(x|z)\right] - D[q(z|x)||p(z)]$$




$q(x)$ ???

$q_\phi(z|x)$ Encoder

$p(z)$ ???

$p_\theta(x|z)$ Decoder

Data distribution,
The expression
above works for a
single x

$$\log p(x) \geq \mathbb{E}_{q(z|x)}\left[\log p(x|z)\right] - D[q(z|x)||p(z)]$$

- Kingma & Welling call p(z) a “prior” and the encoder regularizes q(z|x) to be as close to the prior as possible.
- We’ll see later that it can also be interpreted in terms of *compression*
- By choosing the encoder, prior, we can get an analytic form for this.

$$q_\phi(\tilde{z}_j|x) = \mathcal{N}(z_j; g_{j,\phi}(x), \operatorname{diag} s^2_{j,\phi}(x))$$

$$p(z) = \mathcal{N}(z; 0, \mathbf{I})$$

- KL divergence between two Gaussians is easy.

$$\log p(x) \geq \mathbb{E}_{q(z|x)}\left[\log p(x|z)\right] - D[q(z|x)||p(z)]$$

$$q_\phi(z_j|x) = \mathcal{N}(z_j; g_{j,\phi}(x), \operatorname{diag} s^2_{j,\phi}(x))$$

$$p(z) = \mathcal{N}(z; 0, \mathbf{I})$$

KL divergence between two (diagonal) Gaussians is easy.

$$- \quad 1/2 \sum_{j=1}^{m} \left(1 + \log s_j(x)^2 - g_j(x)^2 - s_j(x)^2\right)$$

$$\log p(x) \geq \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)] - D[q(z|x)||p(z)]$$

How do we do gradients with respect to phi?

# REINFORCE gradient estimator (if time)

$$\nabla_\theta \mathbb{E}_{x \sim p_\theta(x)}[f(x)]$$

$$\nabla_\theta p_\theta(x) = p_\theta(x) \nabla_\theta \log p_\theta(x)$$

From this, we can derive the REINFORCE estimator

$$\nabla_\theta \mathbb{E}_{x \sim p_\theta(x)}[f(x)] = \mathbb{E}_{x \sim p_\theta(x)}[f(x) \nabla_\theta \log p_\theta(x)]$$

$$\log p(x) \geq \mathbb{E}_{q_\phi(z|x)}\left[\log p_\theta(x|z)\right] - D[q(z|x)||p(z)]$$

- How do we do gradients with respect to phi? REINFORCE, or
- We can do the “re-parametrization trick”

$$\mathbb{E}_{q_\phi(z|x)}[h(z)] = \mathbb{E}_{q(\epsilon)}[h(z_\phi(x, \epsilon))]$$

- Let’s look at a concrete example

# Direct example of re-parametrization trick (simplified)

$$\mathbb{E}_{z\sim q_\phi(z|x)}[h(z)] = \mathbb{E}_{\epsilon\sim\mathcal{N}(0,1)}[h(g_\phi(x) + s_\phi(x)\epsilon)]$$

# Direct example of re-parametrization trick

$$\mathbb{E}_{q_\phi(z|x)}[h(z)] = \int dz \frac{e^{-1/2\left(\frac{z-g_\phi(x))}{s_\phi(x)}\right)^2}}{\sqrt{2\pi s_\phi(x)^2}} h(z)$$

Change of variables

$\epsilon = (z - g_\phi(x))/s_\phi(x)$

$z = g_\phi(x) + s_\phi(x)\epsilon$

$d\epsilon = dz/s_\phi(x)$

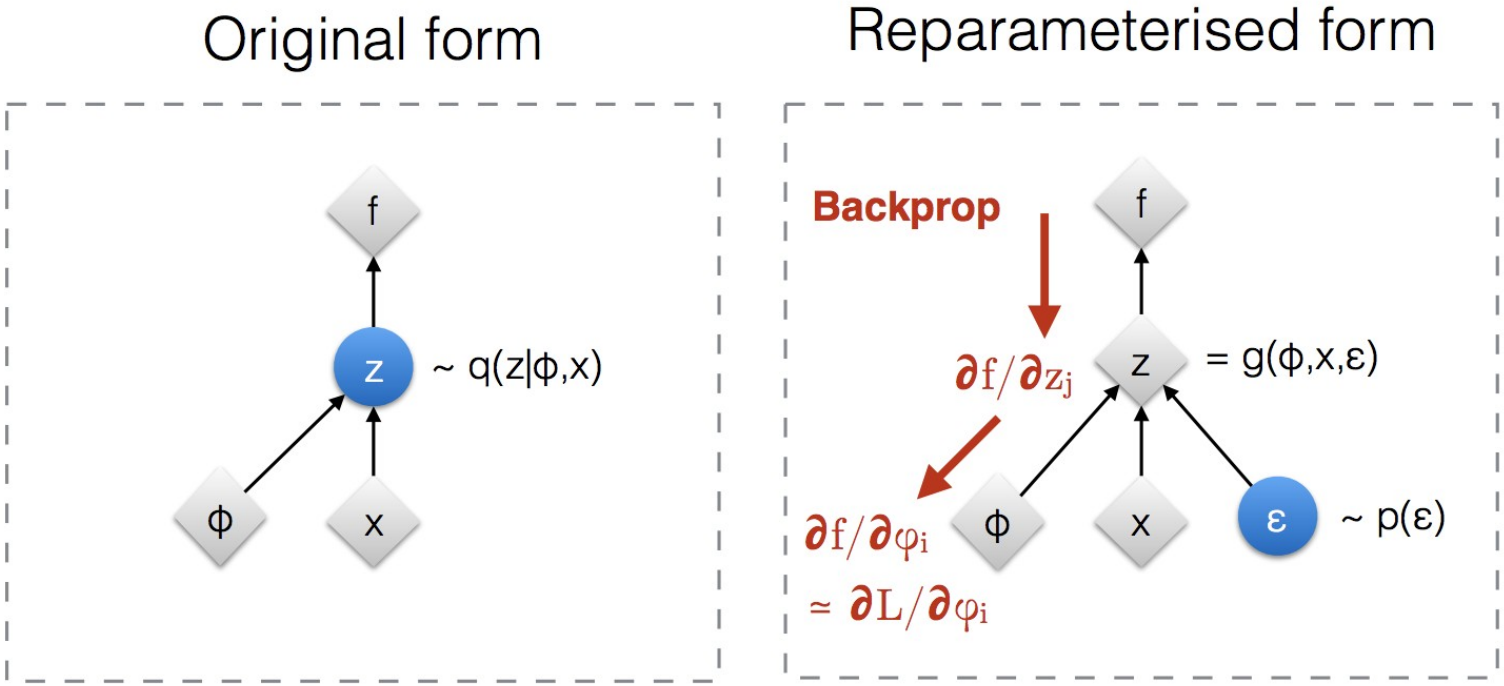
$$= \int d\epsilon \frac{e^{-\epsilon^2/2}}{\sqrt{2\pi}} h(g_\phi(x) + s_\phi(x)\epsilon)$$

$$= \mathbb{E}_{\epsilon\sim\mathcal{N}(0,1)}[h(g_\phi(x) + s_\phi(x)\epsilon)]$$

$$\approx 1/L \sum_{\ell=1}^{L} h(g_\phi(x) + s_\phi(x)\epsilon_\ell)$$

# Visual depiction of re-parametrization

https://lilianweng.github.io/lil-log/2018/08/12/from-autoencoder-to-beta-vae.html




: Deterministic node

: Random node

[Kingma, 2013]
[Bengio, 2013]
[Kingma and Welling 2014]
[Rezende et al 2014]

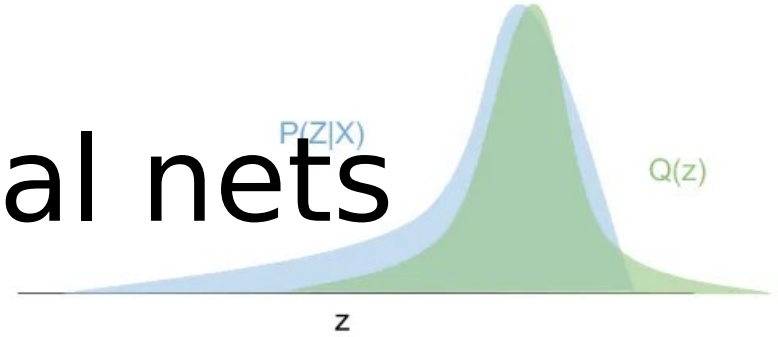
# When can you use re-parametrization trick?

$$\mathbb{E}_{q_\phi(z|x)}[h(z)] = \mathbb{E}_{q(\epsilon)}[h(z_\phi(x, \epsilon))]$$

- You can do this for many (continuous) distributions – Kingma & Welling list examples
- You can even do this with discrete random variables using the Gumbel softmax trick: https://arxiv.org/pdf/1611.01144.pdf

# Implementation details

# Variational inference neural nets




•

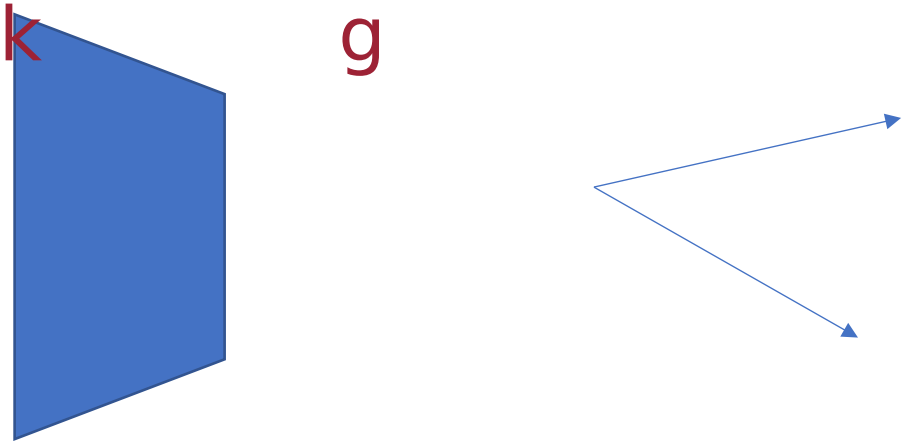
Input

x

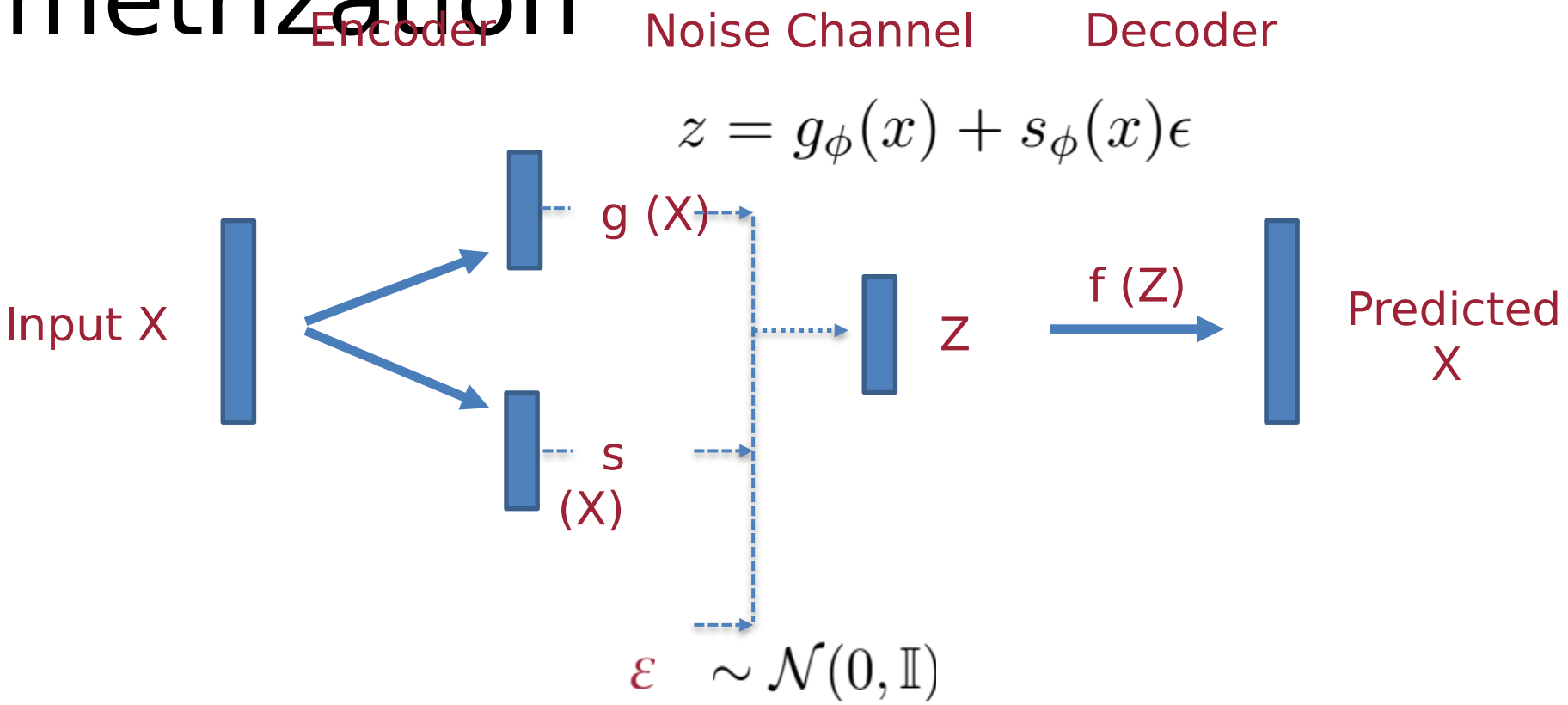
Neural network

Learned embedding

Output mean and (log-variance)



# Visualizing architecture with re-parametrization



# Minimize (Negative) Evidence Lower Bound (ELBO)

$$-\log p(x) \leq -\mathbb{E}_{q(z|x)}\left[\log p(x|z)\right] + D[q(z|x)||p(z)]$$

reconstruction

regularization

$$\approx -\log p(X = x|Z = g_\phi(x) + s_\phi(x)\epsilon) - 1/2 \sum_{j=1}^{m} \left(1 + \log s_j(x)^2 - g_j(x)^2 - s_j(x)^2\right)$$

What *exactly* is term on left?

# Reconstruction with continuous random variables

In that case, a common choice is to use:

$$p(x|z) = \mathcal{N}(f_\theta(z), I)$$

Because then - log p(x|z) becomes:

$$-\log p(x|z) = {}^1\!/_2|x - f(z)|_2^2 + d/2 \log(2\pi)$$

The mean square error!

- f(z) is actually the “mean decoding”
- Nice that we can compare to deterministic autoencoders – same loss plus a regularizer. Also easy to specify in pytorch

# What can we do with it? (computational graph surgery)

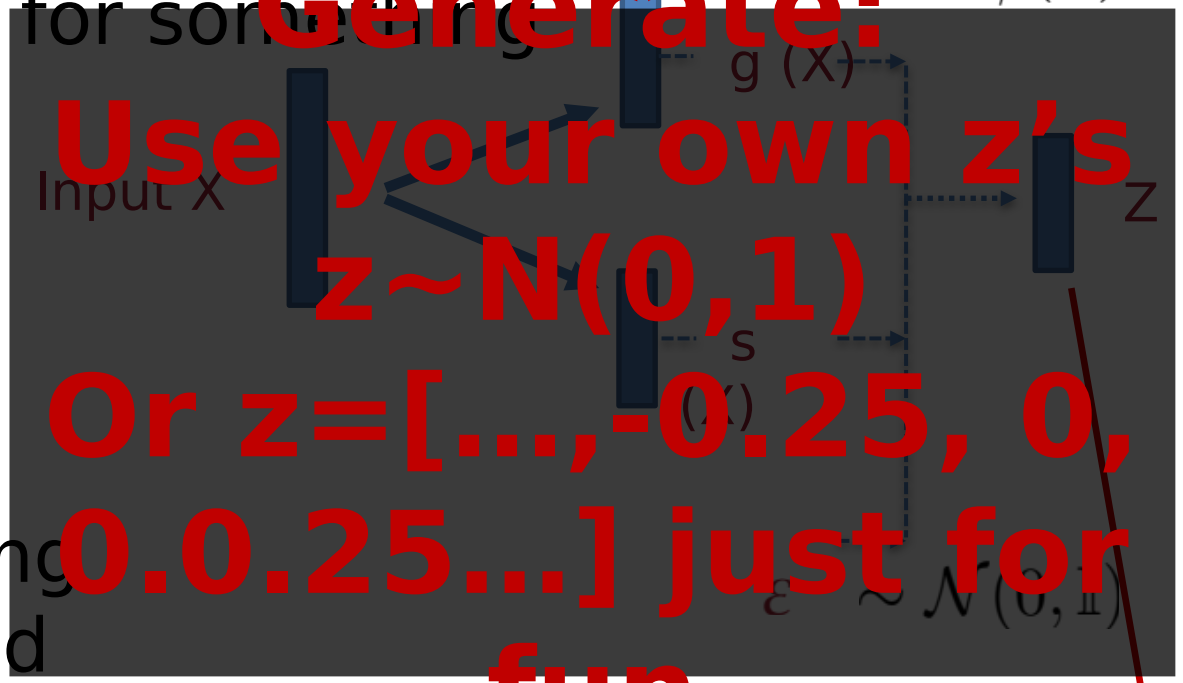
Reconstruction

Estimate likelihood

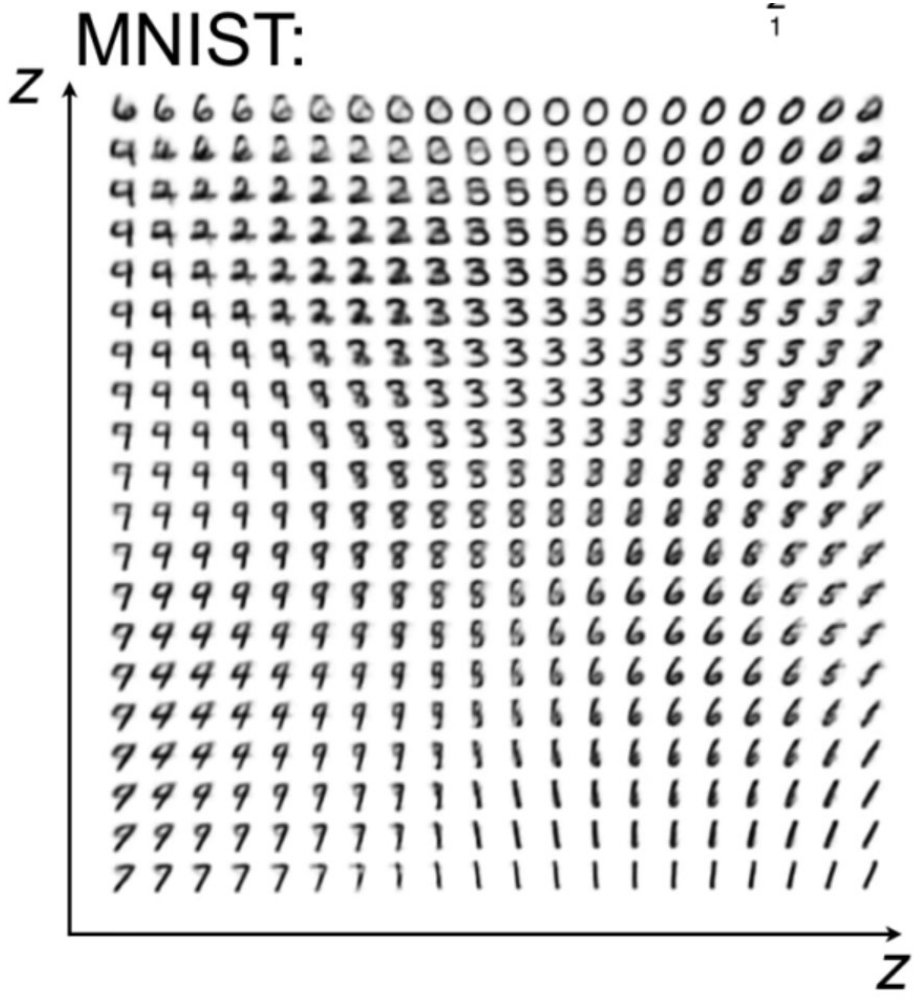
Use z's for something

Sampling

Manifold



# That’s how we get this plot – show the mean reconstruction for each value of z



# VAE Recap

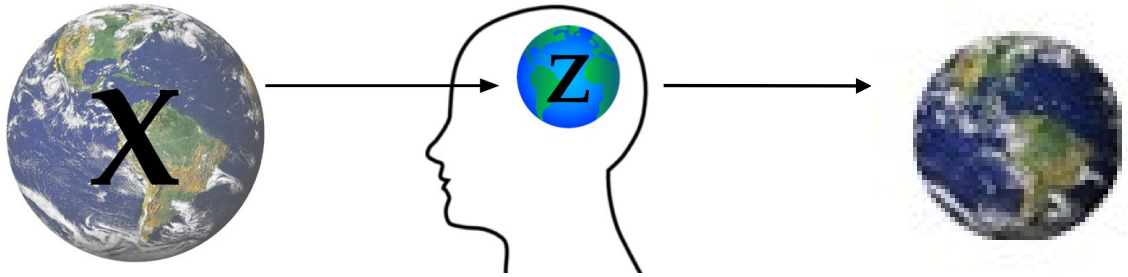
•

$$p(x) = \int dz \; p(x|z)p(z)$$

$$-\log p(x) \leq -\mathbb{E}_{q(z|x)}\left[\log p(x|z)\right] + D[q(z|x)||p(z)]$$

reconstruction

regularization




Z is a compressed *representation* of X, that should be useful for reconstruction, β controls compression

# Rate-distortion / lossy compression formulation of VAE

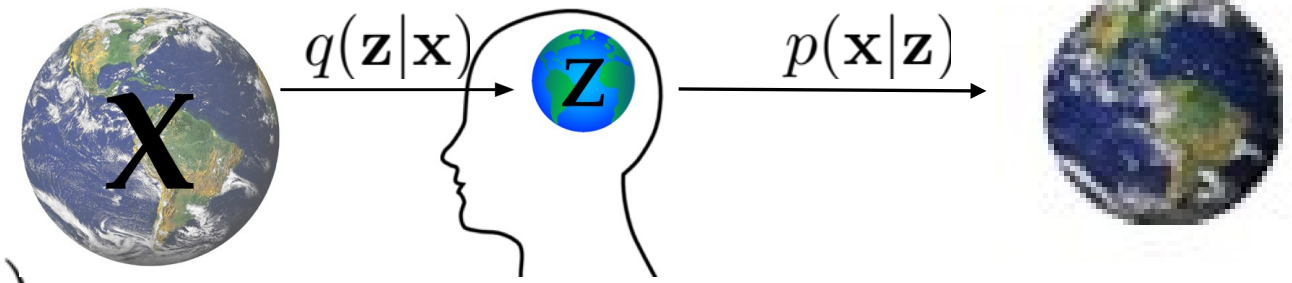
Rate-distortion connection is discussed in several papers:

Alemi, A., Poole, B., Fischer, I., Dillon, J., Saurous, R. A., and Murphy, K. Fixing a broken ELBO. ICML 2018

Rezende, D. J. and Viola, F. Taming VAEs. *arXiv preprint arXiv:1810.00597*, 2018.

Brekelmans, Moyer, Galstyan, and Ver Steeg. "Exact Rate-Distortion in Autoencoders via Echo Noise." *NeurIPS 2019*

# Variational Auto-Encoder (VAE)




$q(\mathbf{x})$

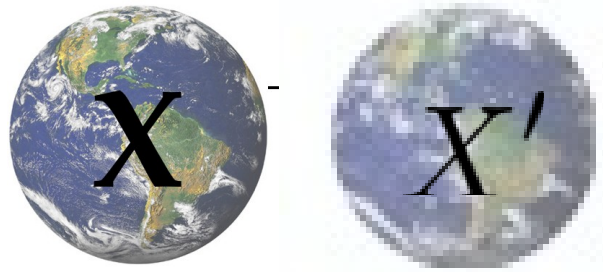
Observed distribution

Encoder (variational approximation to p(z|x))

Decoder (generative model)

- Encoder and decoder parametrized by neural nets
- Goal is to maximize likelihood of observations under generative model

# The Rate-Distortion Problem




d() is a distortion measure on x,x'

- Min rate for given distortion level $D$

$$R(D) = \min I_q(X;Z)$$

$$\text{subject to: } \mathbb{E}[d(x,x')] \leq D$$

- Unconstrained optimization

$$\min I(X;Z) + \beta\mathbb{E}[d(x,x')]$$

- (beta just controls the ratio of these two terms)

# Minimize (Negative) Evidence Lower Bound (ELBO)

$$-\log p(x) \leq -\mathbb{E}_{q(z|x)}\left[\log p(x|z)\right] + \beta D\left[q(z|x)||p(z)\right]$$

Reconstruction ("distortion")

*Rate (upper bound)*

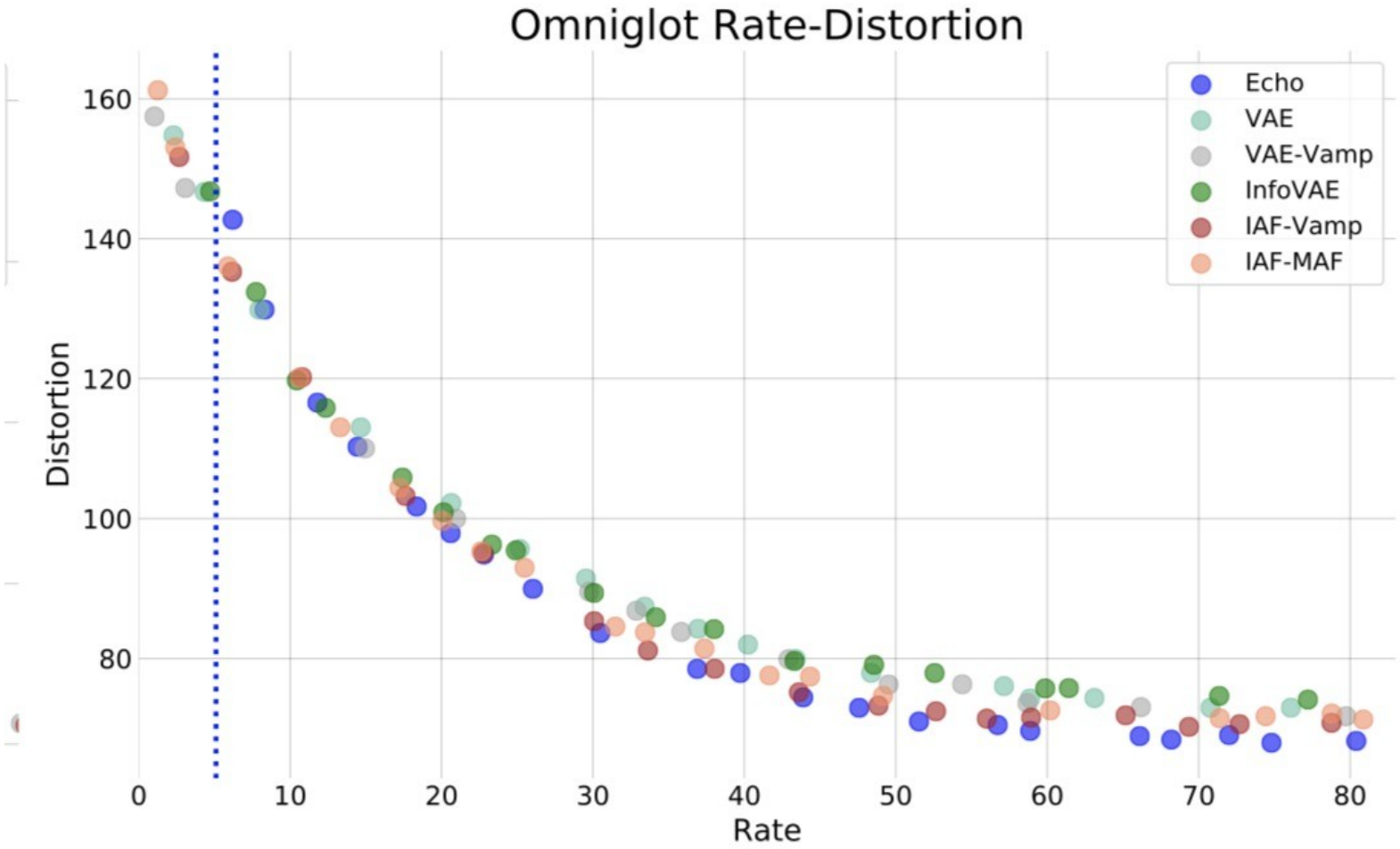
$E_{q(x)}[D[q(z|x)||p(z)]] = E_{q(x)}[D[q(z|x)||q(z)] + D[q(z)||p(z)]] \geq E_{q(x)}[D[q(z|x)||q(z)] = I(z;x)$

Choose $p(z) = q(z) = \int q(z|x)q(x)dx$ for tightest bound

By adding "beta" (beta VAE) you can control the trade-off.

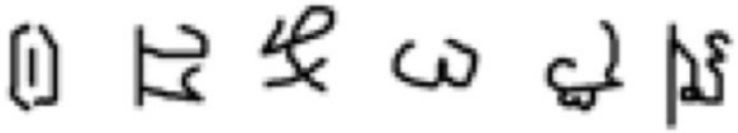
# Rate-distortion curve

- I took this example from Brekelmans et al NeurIPS 2019
- Different points on the curve are achieved by varying beta.
- Beta=1 is special – it gives us the ELBO, a lower bound on log likelihood.
- Achievable points depend on a lot of things – architecture, optimizer, data
- More details like theoretical bounds discussed in Alemi: https://arxiv.org/pdf/1711.00464.pdf



# Visualizing rate-distortion trade-offs on OmniGlot

**Original**

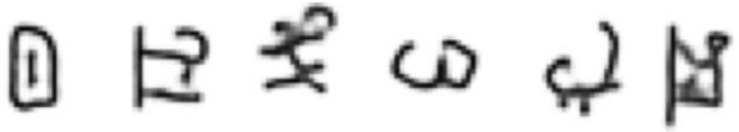


**Echo**

$\beta = 2.0$
R=18.3 D=101.7

$\beta = 1.0$
R=30.5 D=83.6

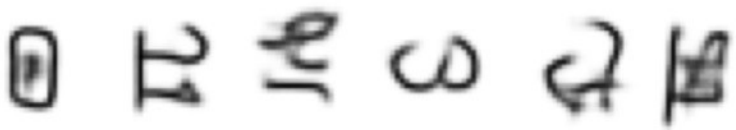
$\beta = 0.1$
R=72.0 D=69.0



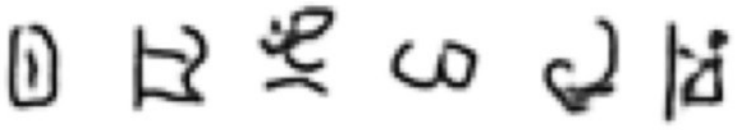
**VAE**

$\beta = 1.75$
R=20.1 D=100.9

$\beta = 1.0$
R=30.1 D=89.4



$\beta = 0.15$
R=71.4 D=74.6



# Vector Quantized-VAE

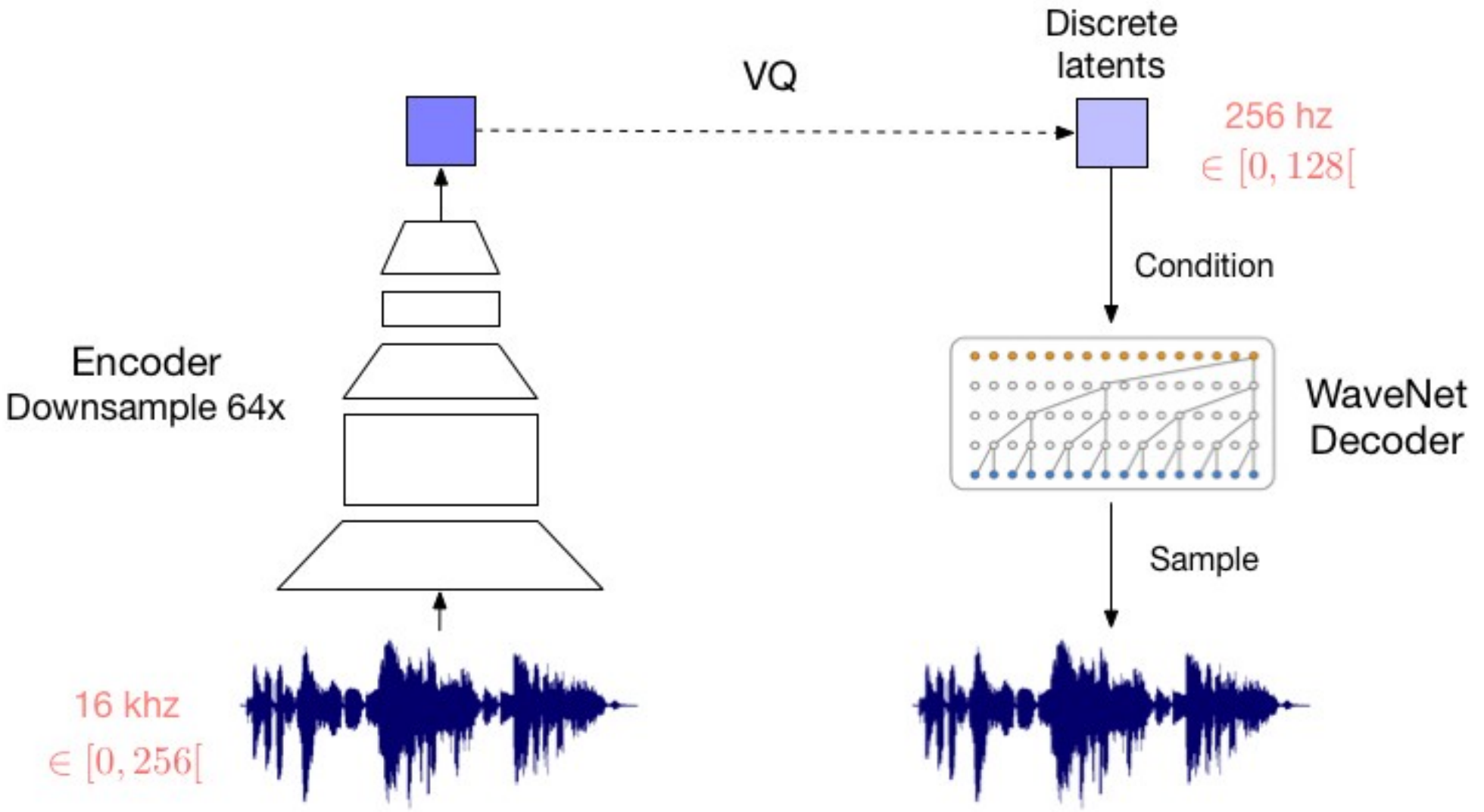
Fixing the rate via *quantization*

https://mlberkeley.substack.com/p/vq-vae

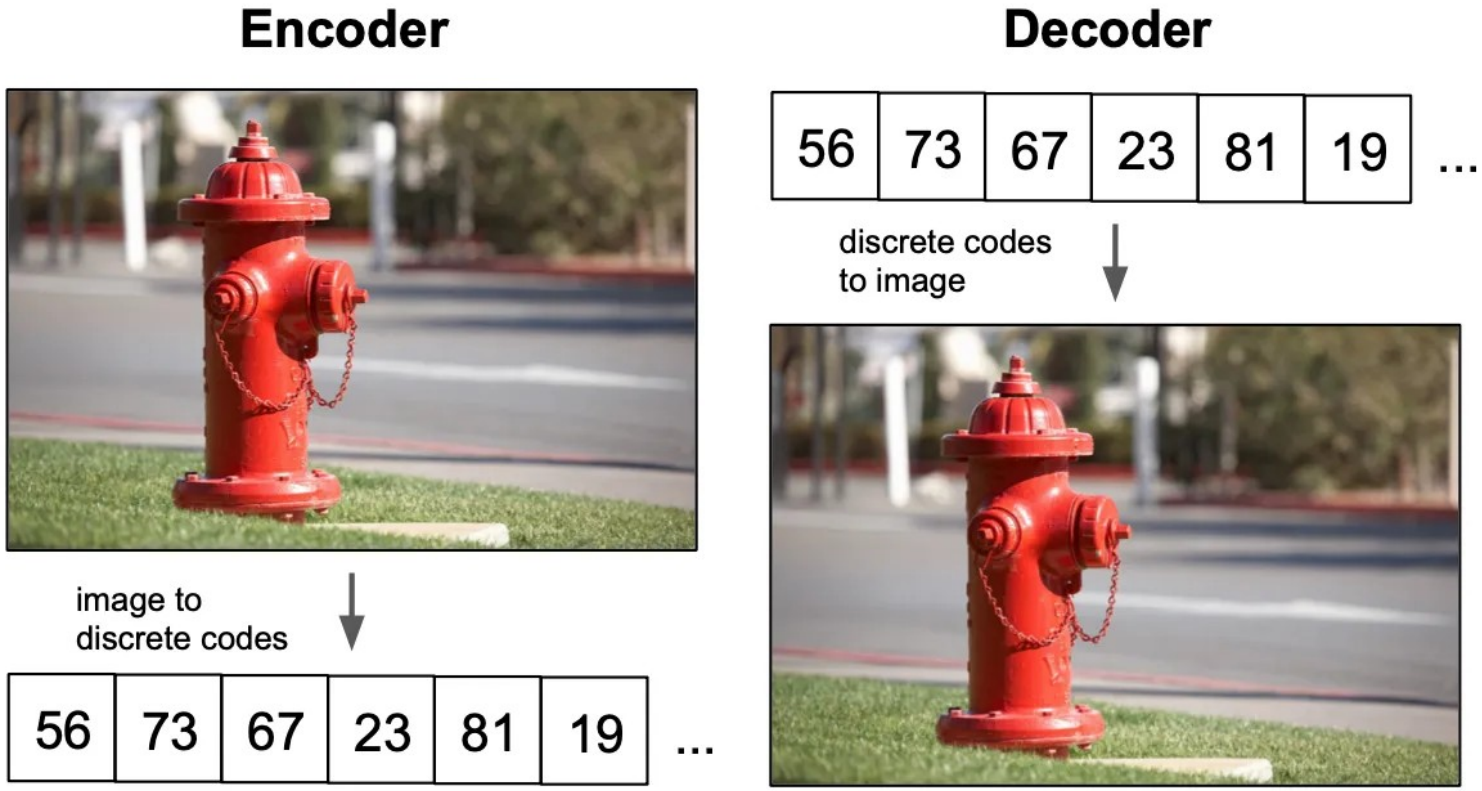
https://avdnoord.github.io/homepage/vqvae/

https://arxiv.org/abs/1711.00937

Discrete ”codebooks” rather than Gaussian “z” is more natural if you want to store a compressed version!



# How do we do this with a VAE?

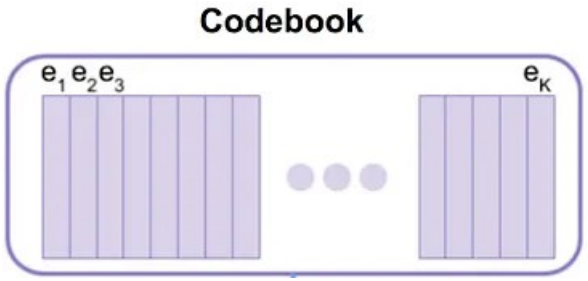



We had “reconstruction” plus “rate”.

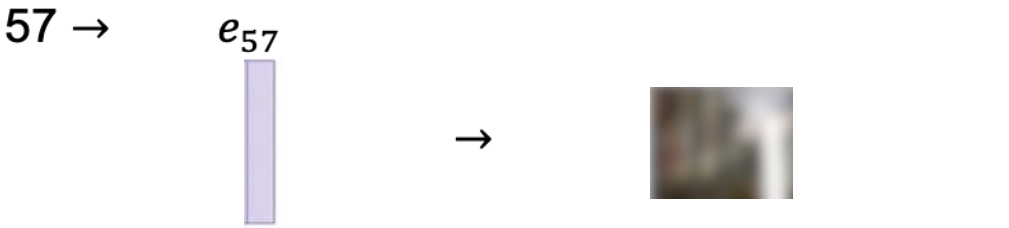
The rate is now capped by the discrete representation!

So we just need to encourage good reconstruction!

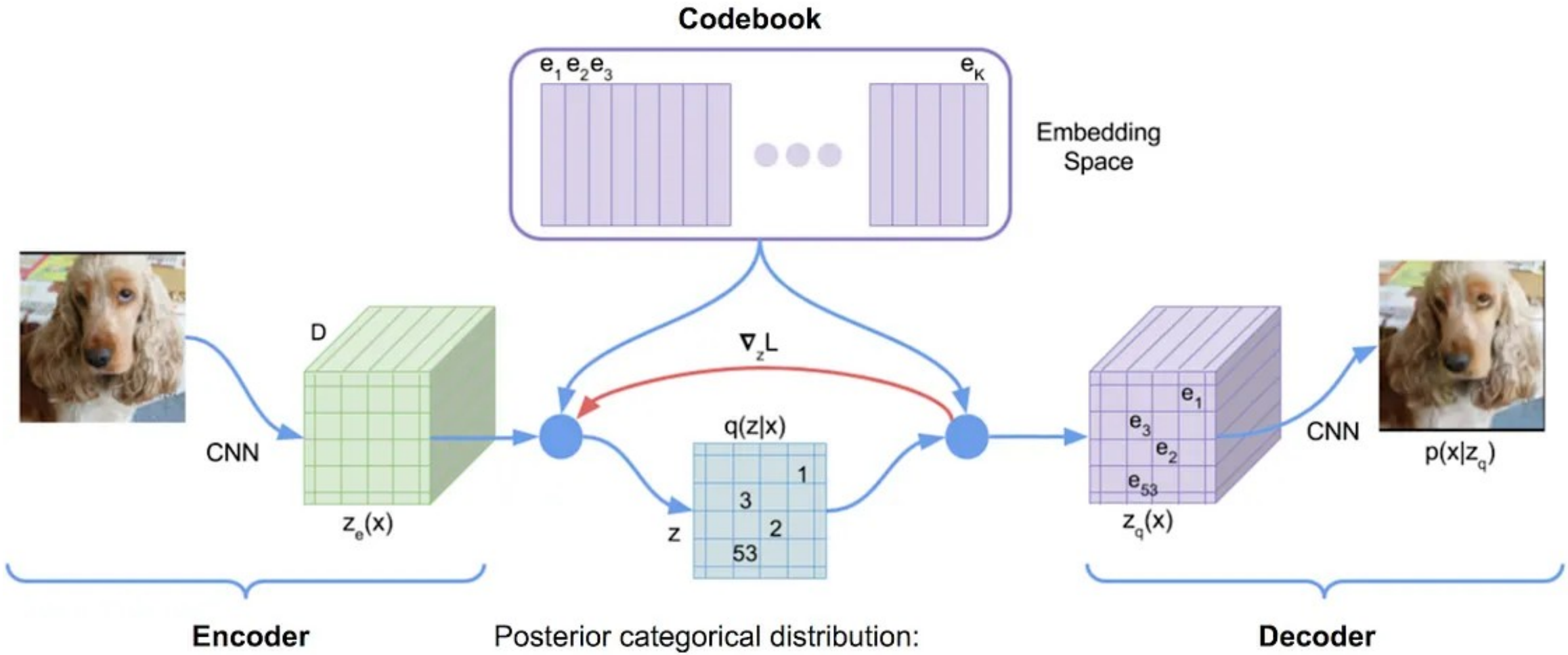
# Codebook


- Find closest vector for this patch
- Encode as the index “57”, e.g.




Reconstructed patch


Codebook
$e_1$ $e_2$ $e_3$
$e_K$
Embedding Space
D
CNN
$z_e(x)$
$\nabla_z L$
q(z|x)
1
3
2
53
z
$e_1$
$e_3$
$e_2$
$e_{53}$
$z_q(x)$
CNN
$p(x|z_q)$
Encoder
Posterior categorical distribution:
Decoder

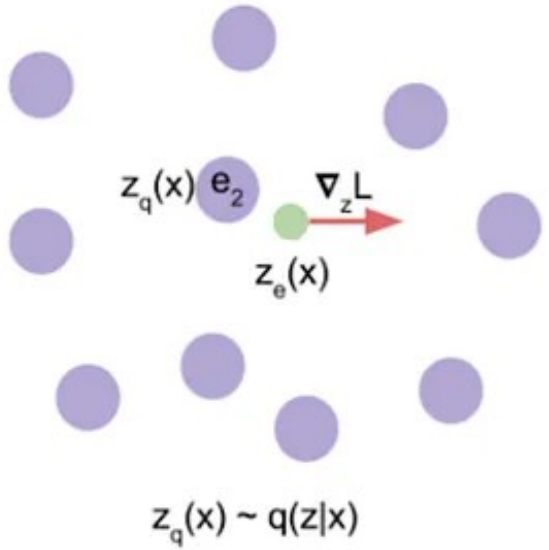
# Loss function for training

$$L = \log p(x|z_q(x)) + \|\mathrm{sg}[z_e(x)] - e\|_2^2 + \beta\|z_e(x) - \mathrm{sg}[e]\|_2^2,$$

Reconstruction

Optimize dictionary, so codes match output of continuous net

Optimize encoder, to give outputs close to codes (“commitment loss”)

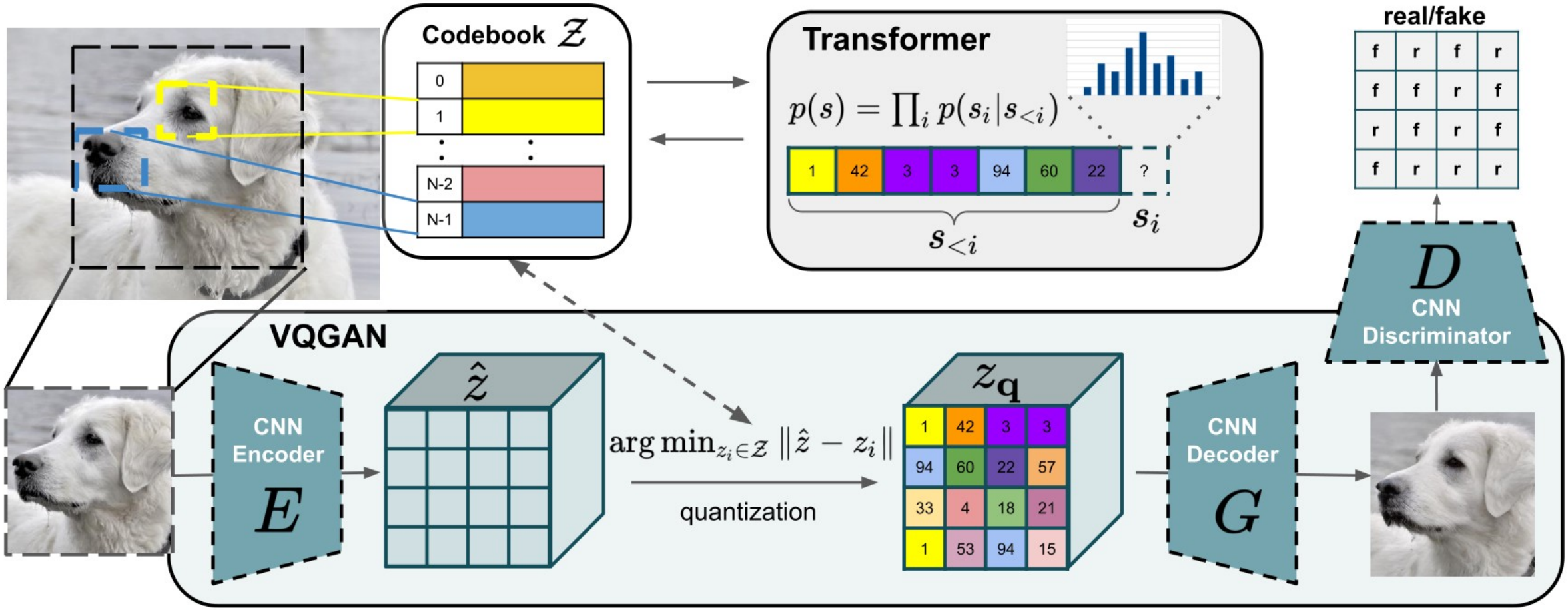



Make sure that continuous embedding is close to discrete embedding

# Vector Quantized: VQ-GAN

https://compvis.github.io/taming-transformers

# Identical to VQ-VAE, but we add the GAN loss as a regularizer!



Monday: Holiday (Memorial day)